

COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S1
DATE PERFORMED:	July 12, 2026	DATE SUBMITTED:	July 14, 2026
NAME:	Mendoza, Ryo Shin P.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 Basic C++ Programming

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

1. Create a function that will take two numbers and display the sum.

```

1  #include <iostream>
2
3  // Function Prototype
4  int sumTwoNum(int first, int last);
5
6  int first, second;
7
8  int main() {
9
10     std::cout << "Enter first number: ";
11     std::cin >> first;
12     std::cout << "Enter second number: ";
13     std::cin >> second;
14     std::cout << "Total: " << sumTwoNum(first,second) << std::endl;
15
16     return 0;
17 }
18
19 int sumTwoNum(int first, int last){
20
21     int total = first + last;
22
23     return total;
24 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Shin\OneDrive\Coding\DSA Activities\HOA 1.1\ILOC> cd "c:\Users\Shin\OneDrive\Coding\DSA Ac
++ NUM1.cpp -o NUM1 } ; if ($?) { .\NUM1 }
Enter first number: 23
Enter second number: 7
Total: 30

```

Figure 1. Source Code and Output

This figure shows the source code and output of the program. The program calculates the two numbers that are entered by the user. Numbers with decimals will not be counted. The output is the total number added.

2. Create a function that will return whether variable A is greater than variable B.

```

1  #include <iostream>
2
3  bool greaterThan(int A, int B);
4
5  int main(){
6
7      int A, B;
8
9      std::cout << "Enter value A: ";
10     std::cin >> A;
11
12     std::cout << "Enter value B: ";
13     std::cin >> B;
14
15     if (greaterThan(A,B)) {
16         std::cout << "A is greater than B" << std::endl;
17     } else {
18         std::cout << "A is not greater than B" << std::endl;
19     }
20
21     return 0;
22 }
23
24 bool greaterThan (int A, int B) {
25
26     return A > B;
27 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Shin\OneDrive\Coding\DSA Activities\HOA 1.1\ILOA> cd "c:\Users\Shin\OneDrive\Coding\DSA Activities\HOA 1.1\ILOA"
++ NUM2.cpp -o NUM2 } ; if ($?) { .\NUM2 }
Enter value A: 12
Enter value B: 6
A is greater than B

```

Figure 2. Source Code and Output

This figure shows the source code and output of the program. The program checks if the value A is greater than value B, thus it will print "A is greater than B". However, if the value B is greater than value A, thus it will print "A is not greater than B".

3. Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

```

3  #include <iostream>
4
5  bool logicalOperators(bool A, bool B);
6
7  int main() {
8
9      bool A, B;
10
11      std::cout << "Enter between 0 and 1" << std::endl;
12      std::cout << "Boolean(A): ";
13      std::cin >> A;
14      std::cout << "Boolean(B): ";
15      std::cin >> B;
16
17      if (logicalOperators(A, B)) {
18          std::cout << "\nLogical operations completed successfully." << std::endl;
19      }
20
21      return 0;
22  }
23
24
25  bool logicalOperators(bool A, bool B) {
26
27      std::cout << "AND (&&): " << (A && B) << std::endl;
28      std::cout << "OR (||): " << (A || B) << std::endl;
29      std::cout << "A NOT(!): " << (!A) << std::endl;
30      std::cout << "B NOT(!): " << (!B) << std::endl;
31
32      return true;
33  }

```

```

Boolean(A): 1
Boolean(B): 0
AND (&&): 0
OR (||): 1
A NOT(!): 0
B NOT(!): 1

Logical operations completed successfully.

```

Figure 3. Source Code and Output

In this figure, it shows the source code and output of the Boolean values program. The program will ask the user to enter two boolean values between 0 and 1 (error will occur if not). The program will output the logical operators and will be successfully completed.

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

Sections	Answer
Header File Declaration Section	<pre>#include <iostream></pre>
Global Declaration Section	Does not have
Class Declaration and Method Definition Section	<pre>class Triangle { private: double totalAngle, angleA, angleB, angleC; public: // Constructor Triangle(double A, double B, double C); // Setter method void setAngles(double A, double B, double C); // Validation method const bool validateTriangle(); };</pre>
Main Function	<pre>int main() { Triangle set1(40, 30, 110); if (set1.validateTriangle()) { std::cout << "The shape is a valid triangle.\n"; } else { std::cout << "The shape is NOT a valid triangle.\n"; } return 0; }</pre>

Method Definition	<pre> Triangle::Triangle(double A, double B, double C) { angleA = A; angleB = B; angleC = C; totalAngle = A + B + C; } // Setter definition void Triangle::setAngles(double A, double B, double C) { angleA = A; angleB = B; angleC = C; totalAngle = A + B + C; } </pre>
-------------------	--

Table 1-1. C++ Structure Code for Answer

Figure 1 shows the implementation and output of the Triangle class program. The program checks whether the three given angles form a valid triangle by verifying that their sum is 180 degrees. For the angles 40°, 30°, and 110°, the program outputs "The shape is a valid triangle."

The program worked as expected and correctly checked whether the given angles form a valid triangle. It added the three angles and verified that their total is 180 degrees. Since the angles 40°, 30°, and 110° add up to 180°, the program displayed that the shape is a valid triangle. This activity helped me to understand how to use classes, constructors, and a number of functions in c++.The program is simple and works correctly, but it can be improved by allowing users to enter different angle values instead of using fixed values in code.

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.

```

2
3  #include <iostream>
4
5  int main() {
6
7      int a = 15;
8      int b = 20;
9      int swap;
10
11     std::cout << "Before swapping: a = " << a << " b = " << b << std::endl;
12
13     swap = a;
14     a = b;
15     b = swap;
16
17     std::cout << "After swapping: a = " << a << " b = " << b << std::endl;
18
19
20     return 0;
21 }

```

```

Before swapping: a = 15 b = 20
After swapping:  a = 20 b = 15

```

Figure 1. Source Code and Output

In this figure, the program asks the user to enter two numbers. It then swaps the values of the two variables using a temporary variable (swap). After swapping, the program displays the new values of the two numbers.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```

2  #include <iostream>
3
4  double kelvinToFahrenheit (double kelvin);
5
6  int main() {
7
8      double kelvin;
9
10     std::cout << "Enter Kelvin to convert in Fahrenheit: ";
11     std::cin >> kelvin;
12
13     std::cout << "Fahrenheit: " << kelvinToFahrenheit(kelvin);
14
15     return 0;
16 }
17
18 double kelvinToFahrenheit (double kelvin) {
19     return (kelvin - 273.15) * 9.0 / 5.0 + 32.0;
20 }

```

```

Enter Kelvin to convert in Fahrenheit: 300
Fahrenheit: 80.33

```

Figure 2. Source Code and Output

The program asks the user to enter a temperature in Kelvin. It uses a function to convert the temperature to Fahrenheit using the conversion formula. After the calculation, the converted temperature is displayed on the screen.

3. Create a C++ program that has a function that will calculate the distance between two points.

```

1  #include <iostream>
2  #include <cmath>
3
4  // Function prototype
5  double calculateDistance(double x1, double y1, double x2, double y2);
6
7  int main() {
8      double x1, y1, x2, y2, distance;
9
10     std::cout << "Enter the coordinates of the first point (x1 y1): ";
11     std::cin >> x1 >> y1;
12
13     std::cout << "Enter the coordinates of the second point (x2 y2): ";
14     std::cin >> x2 >> y2;
15
16     // Function call
17     distance = calculateDistance(x1, y1, x2, y2);
18
19     std::cout << "\nThe distance between the two points is: " << distance << std::endl;
20
21     return 0;
22 }
23
24 // Function definition
25 double calculateDistance(double x1, double y1, double x2, double y2) {
26     return std::sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
27 }

```

```

Enter the coordinates of the first point (x1 y1): 2 3
Enter the coordinates of the second point (x2 y2): 4 5

The distance between the two points is: 2.82843

```

Figure 3. Source Code and Output

The program asks the user to enter the coordinates of two points. It uses a function to calculate the distance between the two points using the distance formula. After the calculation, the program displays the distance between the points.

4. Modify the code given in ILO B and add the following functions:
- A function to compute for the area of a triangle
 - A function to compute for the perimeter of a triangle
 - A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
#include <iostream>
#include <cmath>

class triangle {
private:
    double sideA, sideB, sideC;
    double angA, angB, angC;

public:
    triangle(double A, double B, double C,
             double sA, double sB, double sC);

    bool validateTriangle();
    double area();
    double pmtr();
    void trityp();
};

int main() {
    double angA, angB, angC;
    double sideA, sideB, sideC;
    int choice;

    std::cout << "Enter Angle A: ";
    std::cin >> angA;

    std::cout << "Enter Angle B: ";
    std::cin >> angB;

    std::cout << "Enter Angle C: ";
    std::cin >> angC;
```

```
std::cout << "Enter Side A: ";
std::cin >> sideA;

std::cout << "Enter Side B: ";
std::cin >> sideB;

std::cout << "Enter Side C: ";
std::cin >> sideC;

triangle tri(angA, angB, angC, sideA, sideB, sideC);

if (!tri.validateTriangle()) {
    std::cout << "\nThis is NOT a valid triangle.\n";
    return 0;
}

std::cout << "\n==== MENU =====\n";
std::cout << "1. Compute Area\n";
std::cout << "2. Compute Perimeter\n";
std::cout << "3. Determine Triangle Type\n";
std::cout << "Enter Choice: ";
std::cin >> choice;

switch (choice) {
    case 1:
        std::cout << "Area = " << tri.area() << std::endl;
        break;

    case 2:
        std::cout << "Perimeter = " << tri.pmtr() << std::endl;
        break;

    case 3:
        tri.trityp();
        break;

    default:
```

```
        std::cout << "Invalid Choice.\n";
    }

    return 0;
}

// Constructor
triangle::triangle(double A, double B, double C,
                    double sA, double sB, double sC) {

    angA = A;
    angB = B;
    angC = C;

    sideA = sA;
    sideB = sB;
    sideC = sC;
}

// Validate Triangle
bool triangle::validateTriangle() {

    if (angA <= 0 || angB <= 0 || angC <= 0)
        return false;

    if (sideA <= 0 || sideB <= 0 || sideC <= 0)
        return false;

    if (std::fabs((angA + angB + angC) - 180.0) > 0.001)
        return false;

    if (sideA + sideB <= sideC)
        return false;

    if (sideA + sideC <= sideB)
        return false;

    if (sideB + sideC <= sideA)
```

```

        return false;

    return true;
}

// Area using Heron's Formula
double triangle::area() {
    double s = (sideA + sideB + sideC) / 2.0;
    return std::sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}

// Perimeter
double triangle::pmtr() {
    return sideA + sideB + sideC;
}

// Determine Triangle Type
void triangle::trityp() {

    if (angA < 90 && angB < 90 && angC < 90)
        std::cout << "The triangle is Acute-Angled.\n";

    else if (angA > 90 || angB > 90 || angC > 90)
        std::cout << "The triangle is Obtuse-Angled.\n";

    else
        std::cout << "The triangle is Others (Right Triangle).\n";
}

```

Figure 4. Source Code

The program uses a triangle class to store the angles and side lengths of a triangle. It first checks if the triangle is valid. If it is valid, the user can choose to compute the area, compute the perimeter, or determine the type of the triangle. The area is calculated using Heron's Formula, the perimeter is found by adding the three sides, and the triangle type is determined by checking its angles.

```
Enter Angle B: 60
Enter Angle C: 60
Enter Side A: 5
Enter Side B: 5
Enter Side C: 5

===== MENU =====
1. Compute Area
2. Compute Perimeter
3. Determine Triangle Type
Enter Choice: 1
Area = 10.8253
```

Figure 5. Output

The program asks the user to enter the angles and side lengths of a triangle. If the values do not form a valid triangle, it displays an error message. If the triangle is valid, it shows a menu where the user can choose to calculate the area, calculate the perimeter, or identify whether the triangle is acute-angled, obtuse-angled, or a right triangle. The program then displays the corresponding result.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT, Quillbot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: For grammar correction, idea generator, code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I affirm that I using AI did not change my critical thinking, and the idea for the code.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



COMPUTER
ENGINEERING

QUEZON CITY

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.